



CIN0136

Desenvolvimento de Software

Desenvolvimento de Software

Prof. Vinicius Cardoso Garcia

vcg@cin.ufpe.br :: @vinicius3w :: assertlab.com

<http://viniciusgarcia.me>



**Centro de
Informática**
UFPE



UNIVERSIDADE
FEDERAL
DE PERNAMBUCO

Licença do material

Este Trabalho foi licenciado com uma Licença



**Atribuição-NãoComercial-
Compartilha Igual 4.0 Internacional
(CC BY-NC-SA 4.0)**

Mais informações visite

<https://creativecommons.org/licenses/by-nc-sa/4.0/deed.pt> BR



Engenharia de Software em Dimensões

É um material abrangente, didático e atualizado que apresenta a Engenharia de Software como uma disciplina multidimensional. Cada capítulo foca em uma dimensão prática, conectando teoria e realidade profissional com clareza, profundidade e propósito.

Prof. Vinicius Cardoso Garcia

<https://bit.ly/esdbook>

Engenharia de Software em Dimensões

Da base conceitual à prática profissional



Ponto de Commit

O podcast figital da disciplina (IF977) Engenharia de Software do curso de bacharelado em Sistemas de Informação do Centro de Informática (CIn) da Universidade Federal de Pernambuco (UFPE).

<https://go.uaite.xyz/ponto-de-commit>

Ponto de Commit



O podcast figital da disciplina de Engenharia de Software



Engenharia de Software Moderna

Engenharia de Software Moderna é um livro destinado a alunos de cursos de graduação em Computação. Ele pode ser lido também por profissionais que buscam um conhecimento horizontal da área.

Prof. Marco Tulio Valente

<https://engsoftmoderna.info>

@engsoftmoderna

ENGENHARIA — DE — SOFTWARE MODERNA

Princípios e práticas para
desenvolvimento de software
com produtividade



MARCO TULIO VALENTE





Orquestrando Serviços com Docker Compose

Você já sabe subir um container. Agora é hora de fazer vários serviços conversarem entre si (frontend, backend e banco de dados) como uma orquestra afinada. Neste módulo, você vai dominar o Docker Compose do zero ao deploy.



Agenda

01

O limite do Docker isolado

Por que rodar containers manualmente não escala para sistemas reais com múltiplos serviços.

03

Projeto PetFood completo

Um docker-compose.yml real para React + Express + PostgreSQL, comentado linha a linha.

02

Anatomia do docker-compose.yml

Cada diretiva explicada: services, build, ports, volumes, depends_on e muito mais.

04

Variáveis de ambiente e deploy

Padrão .env, segurança, e como plataformas como Railway e Render funcionam por baixo dos panos.



O Limite do Docker Isolado

Rodar um único container funciona bem para experimentos. Mas aplicações reais são compostas por múltiplos serviços que precisam se comunicar, compartilhar redes e respeitar ordem de inicialização.

🔧 Processo manual

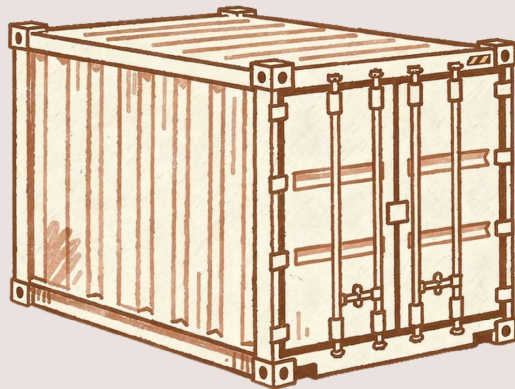
Para cada serviço: `docker build`, `docker network create`, `docker run` com dezenas de flags. Difícil de reproduzir e manter.

🔗 Dependências ignoradas

Nada garante que o banco suba antes do backend. Erros de conexão aparecem na hora menos esperada.

📋 Zero documentação executável

Um README com 20 comandos não é confiável. O `docker-compose.yml` é a documentação e o script ao mesmo tempo.



Docker Compose ao Resgate

Com Docker puro

```
docker network create app-net

docker run -d \
  --name postgres \
  --network app-net \
  -e POSTGRES_PASSWORD=secret \
  postgres:15

docker run -d \
  --name backend \
  --network app-net \
  -p 3000:3000 \
  my-backend
```

Com Docker Compose

```
docker compose up
```

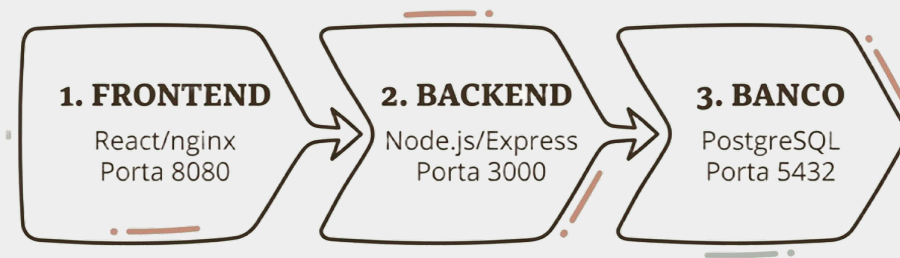
Um único comando sobe todos os serviços, cria a rede automaticamente, respeita a ordem de dependências e exibe os logs unificados. Simples assim.

- ✓ O Compose transforma um processo frágil e manual em algo repetível, versionável e legível por qualquer membro do time.



O PetFood não é um serviço — são três

O sistema PetFood é composto por três containers independentes que precisam se comunicar. Subir cada um manualmente exige múltiplos terminais, longos comandos docker run, criação manual de redes e passagem explícita de variáveis, para cada reinicialização.



⚠ Para subir os três com Docker puro: quantos terminais? Quantos comandos? Quantos parâmetros para memorizar?



Um arquivo. Um comando. Um ambiente.

O Docker Compose substitui uma sequência de comandos manuais por um único arquivo declarativo. Em vez de memorizar flags, redes e variáveis, você descreve o estado desejado, e o Compose faz acontecer.

Sem Compose

```
docker network create petfood-net
docker run --network petfood-net \
  -e POSTGRES_DB=petfood postgres
docker run --network petfood-net \
  -e DATABASE_URL=... backend
docker run --network petfood-net \
  -p 8080:80 frontend
```

Com Compose

```
docker compose up
```

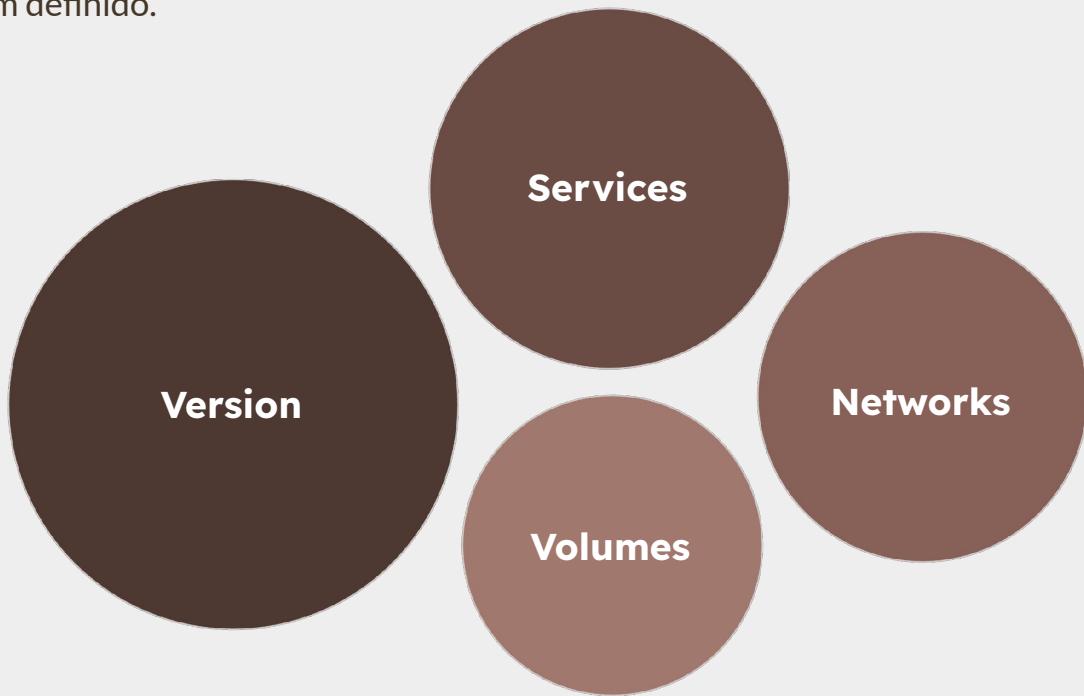
Todos os serviços sobem, a rede é criada automaticamente, variáveis são injetadas, e a ordem de inicialização é respeitada.

O `docker-compose.yml` é o manual de montagem do seu ambiente. Quem lê esse arquivo sabe exatamente como o sistema funciona.



Anatomia do docker-compose.yml

O arquivo é escrito em YAML e possui uma estrutura hierárquica clara. Cada chave tem um propósito bem definido.



Nas próximas seções vamos dissecar cada uma dessas camadas com exemplos reais do projeto PetFood.

A estrutura do arquivo

O `docker-compose.yml` segue uma estrutura hierárquica clara. Cada serviço declara como seu container deve ser construído, configurado e conectado aos demais. O Compose cria automaticamente uma rede interna — todos os serviços se enxergam pelo nome, sem configuração adicional.

```
services:                # os containers que compõem o sistema
  nome-servico:
    image: ...           # imagem pronta do Docker Hub
    build: ...           # ou: Dockerfile local
    ports: ...           # mapeamento host:container
    environment:         # variáveis de ambiente
      CHAVE: valor
    depends_on:          # ordem de inicialização
      - outro-servico
    volumes:              # persistência de dados
      - nome:/caminho

volumes:                 # volumes nomeados (dados que sobrevivem ao down)
```

❗ O Compose cria uma rede interna automaticamente, todos os serviços se enxergam pelo nome do serviço, não por IP.



Diretiva: `services`

O bloco `services` é o coração do arquivo. Cada chave dentro dele define um serviço — que se tornará um container em execução.

```
services:
  frontend: # nome do serviço (também vira hostname na rede)
    build: ./frontend
    ports:
      - "80:80"

  backend:
    build: ./backend
    ports:
      - "3000:3000"

  db:
    image: postgres:15
```

- ❗ O nome do serviço (`frontend`, `backend`, `db`) é usado como hostname DNS interno. Outros containers podem se conectar usando exatamente esse nome.



image vs build

image: usar uma imagem pronta

```
services:
  db:
    image: postgres:15

  cache:
    image: redis:7-alpine
```

Use quando a imagem já existe no Docker Hub ou em um registry privado. Ideal para serviços de infraestrutura como bancos de dados e caches.

build: construir localmente

```
services:
  backend:
    build:
      context: ./backend
      dockerfile: Dockerfile

  frontend:
    build: ./frontend
    # shorthand: context=./frontend
```

Use quando você tem o código-fonte e precisa gerar a imagem. O Compose chama docker build automaticamente antes de subir o container.



ports vs expose

ports: publicar para o host

```
"HOST:CONTAINER"
- "80:80"
- "3000:3000"
```

Mapeia uma porta do seu computador para o container. Qualquer processo no host (e na internet, se houver firewall aberto) pode acessar.

expose: apenas entre serviços

```
expose:
- "5432"
```

Documenta e libera a porta *somente dentro da rede interna do Compose*. O banco de dados não fica exposto para fora, mas o backend consegue conectar normalmente.

⚠ Nunca exponha a porta do PostgreSQL (5432) para o host em produção. Use `expose` para mantê-la privada à rede interna do Compose.



environment — Variáveis de Ambiente

Configura variáveis de ambiente dentro do container. Existem duas formas equivalentes:

Forma de lista

```
environment:  
  - POSTGRES_USER=petfood  
  - POSTGRES_PASSWORD=secret  
  - POSTGRES_DB=petfooddb
```

Forma de mapa (recomendada)

```
environment:  
  POSTGRES_USER: petfood  
  POSTGRES_PASSWORD: secret  
  POSTGRES_DB: petfooddb
```

Para não deixar senhas no arquivo versionado, use referências ao arquivo .env:

```
environment:  
  POSTGRES_PASSWORD: ${DB_PASSWORD} # lido do .env automaticamente
```





depends_on — Ordem de Inicialização

Define que um serviço só deve iniciar *depois* de outro. Evita que o backend tente conectar ao banco antes de ele estar pronto.

```
services:
  backend:
    build: ./backend
    depends_on:
      db:
        condition: service_healthy # aguarda o healthcheck passar

  db:
    image: postgres:15
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U petfood"]
      interval: 5s
      timeout: 5s
      retries: 5
```

❗ `service_started` aguarda o container subir. `service_healthy` é mais robusto e aguarda o healthcheck passar, preferível para bancos de dados.



volumes — Persistência de Dados

Volume nomeado (produção)

```
services:
  db:
    image: postgres:15
    volumes:
      - pgdata:/var/lib/postgresql/data

volumes:
  pgdata:  # declarado aqui embaixo
```

Gerenciado pelo Docker. Os dados sobrevivem ao docker compose down. Ideal para bancos em produção.

Bind mount (desenvolvimento)

```
services:
  backend:
    build: ./backend
    volumes:
      - ./backend:/app
      - /app/node_modules
```

Mapeia uma pasta do seu computador diretamente. Alterações no código são refletidas em tempo real dentro do container, perfeito para hot reload no desenvolvimento.



O DNS Interno do Compose

Um dos superpoderes menos conhecidos do Compose: **o nome do serviço vira o hostname** na rede interna. Nenhuma configuração extra é necessária.



Conexão do backend ao banco

Em vez de usar `localhost` ou um IP, use o nome do serviço:

```
postgresql://db:5432/petfooddb
```



Rede criada automaticamente

O Compose cria uma rede bridge privada para todos os serviços do arquivo. Sem

```
docker network create
```

 manual.

Isolamento por padrão

Serviços de projetos diferentes ficam isolados em redes separadas. Sem colisão de nomes entre projetos distintos.



Apresentando o Sistema PetFood

O PetFood é um e-commerce de ração para pets com arquitetura de três camadas: interface em React servida pelo nginx, API REST em Node.js/Express e banco de dados PostgreSQL. É o projeto que usaremos para montar um `docker-compose.yml` completo e comentado.

Frontend

React + Vite buildado e servido pelo nginx na porta 80

Backend

Node.js + Express, API REST na porta 3000

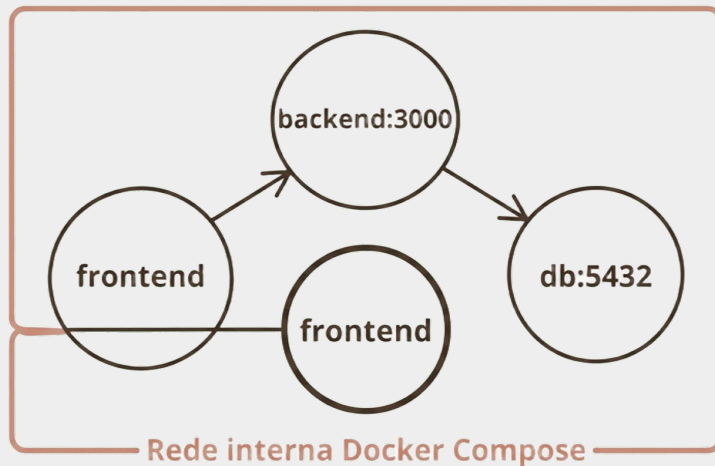
Database

PostgreSQL 15 com volume persistente



Nome do serviço = hostname

O Docker Compose implementa um DNS interno automático. Cada serviço declarado no arquivo se torna um hostname resolvível pelos demais serviços na mesma rede, sem nenhuma configuração adicional. Isso tem uma consequência direta e importante na sua `DATABASE_URL`.



Nomes resolvidos automaticamente sem configuração adicional.

Fora do Compose

Você usa `localhost:5432` para conectar ao banco rodando na sua máquina.

Dentro do Compose

Você usa `db:5432`, o nome do serviço. Usar localhost aqui fará a conexão falhar silenciosamente.



docker-compose.yml — Parte 1: Cabeçalho e Frontend

```
# PetFood – docker-compose.yml
# Sobe frontend (React/nginx), backend (Express) e banco (PostgreSQL)
services:
  # — FRONTEND —
  frontend:
    build:
      context: ./frontend      # pasta com o Dockerfile do React
    ports:
      - "8080:80"             # acessível em http://localhost
    depends_on:
      - backend                # sobe somente após o backend iniciar
    environment:
      - VITE_API_URL=http://localhost:3000
```

❑ O nginx dentro do container do frontend serve os arquivos estáticos buildados e faz proxy das chamadas `/api/*` para o serviço `backend` usando o DNS interno do Compose.

✅ Sistema completo em uma linha de fluxo:

`http://localhost:8080` → frontend → `http://backend:3000` → banco



docker-compose.yml — Parte 2: Backend

```
# — BACKEND —————
backend:
  build:
    context: ./backend      # pasta com o Dockerfile do Express
  ports:
    - "3000:3000"          # expõe a API para o host (útil no dev)
  depends_on:
    db:
      condition: service_healthy # aguarda o postgres estar pronto
  environment:
    NODE_ENV: production
    DATABASE_URL: postgresql://${DB_USER}:${DB_PASSWORD}@db:5432/${DB_NAME}
    # 'db' é o hostname resolvido pelo DNS interno do Compose ^^^
    PORT: 3000
  env_file:
    - .env                  # carrega variáveis do arquivo .env local
```

❗ Perceba que a DATABASE_URL aponta para **db**, o nome exato do serviço PostgreSQL definido abaixo. Isso é o DNS interno do Compose em ação.



docker-compose.yml — Parte 3: Banco de Dados

```
# — DATABASE —————
db:
  image: postgres:15-alpine # imagem oficial, versão leve
  expose:
    - "5432" # visível apenas internamente (não no host)
  environment:
    POSTGRES_USER: ${DB_USER}
    POSTGRES_PASSWORD: ${DB_PASSWORD}
    POSTGRES_DB: ${DB_NAME}
  volumes:
    - pgdata:/var/lib/postgresql/data # dados persistentes
    - ./db/init.sql:/docker-entrypoint-initdb.d/init.sql # seed inicial
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U ${DB_USER}"]
    interval: 5s
    timeout: 5s
    retries: 5
```

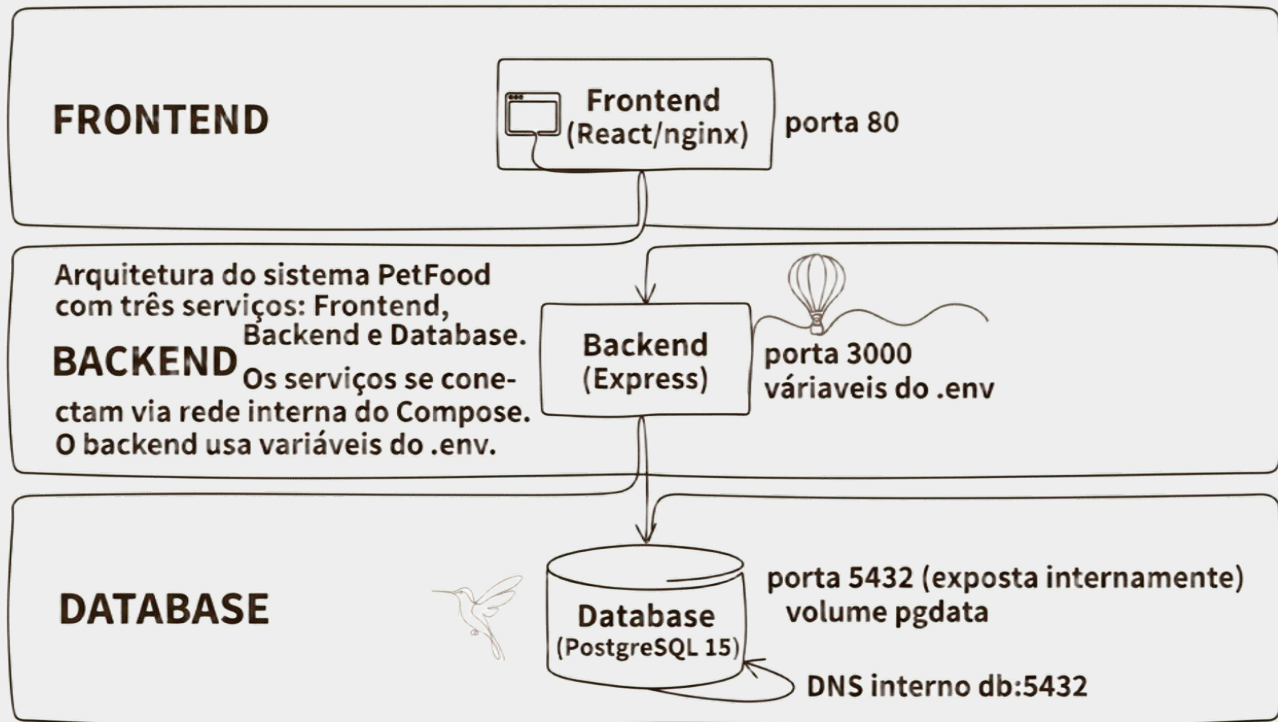
```
# — VOLUMES NOMEADOS —————
volumes:
  pgdata: # declaração obrigatória para volumes nomeados
```

🔔  **expose vs ports:** `expose` abre a porta apenas para a rede interna do Compose. O banco não fica exposto para o mundo, só o backend o acessa.

⚠️  Sem o volume `db_data`, todos os dados são perdidos a cada `docker compose down`. O volume garante persistência entre reinicializações.



O Arquivo Completo — Visão Geral



• O Compose gerencia toda essa complexidade com um único arquivo declarativo. A ordem de boot, a rede interna, os volumes e as variáveis de ambiente são todos orquestrados automaticamente.

Subindo e Gerenciando os Serviços

`docker compose up -d`

Sobe todos os serviços em background (detached). Use `--build` para forçar rebuild das imagens antes de subir.

`docker compose logs -f backend`

Acompanha os logs de um serviço específico em tempo real. Omita o nome para ver todos os serviços juntos.

`docker compose ps`

Lista o status de todos os containers do projeto: nome, estado, portas mapeadas.

`docker compose exec backend sh`

Abre um shell interativo dentro de um container em execução. Útil para debug e inspeção.



Stop vs Down vs Down -v

Três comandos que parecem iguais mas têm efeitos muito diferentes. Saber a diferença evita dores de cabeça, especialmente em produção.

`docker compose stop`

Para os containers sem removê-los. Dados e estado são preservados. Use quando quiser pausar temporariamente e retomar com `docker compose start`.

`docker compose down`

Para e remove os containers e as redes criadas. Os volumes nomeados são mantidos, seus dados do banco continuam intactos.

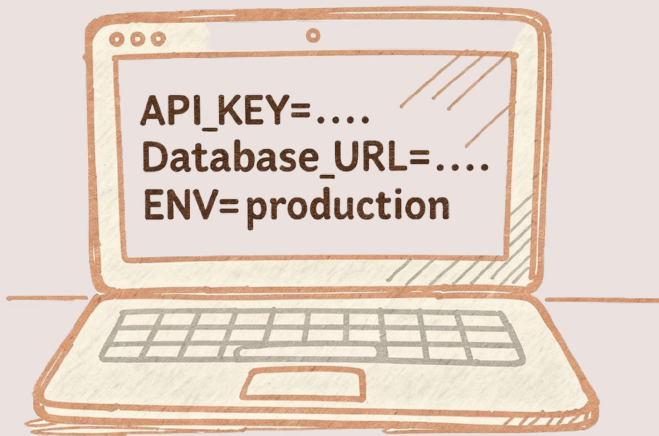
`docker compose down -v`

Remove tudo, incluindo os volumes nomeados. O banco de dados é apagado permanentemente. Use com cautela, ideal para resetar o ambiente de dev.

⊗ Nunca execute `docker compose down -v` em produção sem backup. Este comando é destrutivo e irreversível para os dados persistidos em volumes.



Variáveis de Ambiente: Padrão .env + .env.example



Senhas e segredos não devem jamais ser commitados no Git.
O padrão da indústria é usar dois arquivos complementares.

.env (nunca commitar)

```
DB_USER=petfood
DB_PASSWORD=minha_senha_secreta
DB_NAME=petfooddb
JWT_SECRET=abc123supersecret
```

Adicione ao `.gitignore` imediatamente.
Contém os valores reais usados
localmente.

.env.example (sempre commitar)

```
DB_USER=
DB_PASSWORD=
DB_NAME=
JWT_SECRET=
```

Template público com as chaves necessárias,
mas sem valores sensíveis. Novos membros
do time copiam e preenchem com seus
próprios valores.



Usando Variáveis no `docker-compose.yml`

O Compose carrega o arquivo `.env` da pasta atual automaticamente. Você pode referenciar qualquer variável com a sintaxe `${VARIABLE}`.

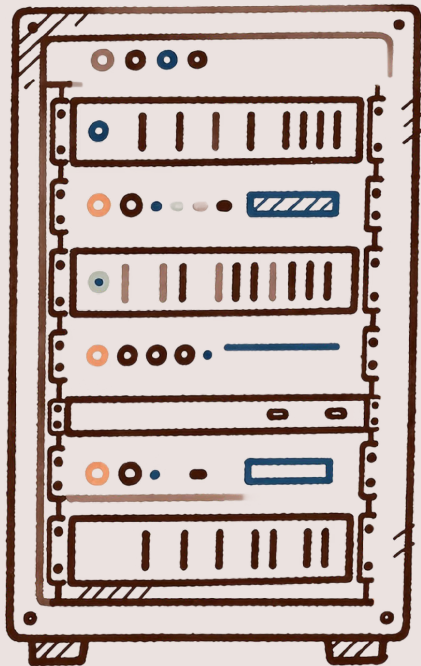
```
# .env
DB_USER=petfood
DB_PASSWORD=s3cr3t
DB_NAME=petfooddb
BACKEND_PORT=3000

# docker-compose.yml
services:
  backend:
    ports:
      - "${BACKEND_PORT}:3000" # porta configurável pelo .env
    environment:
      DATABASE_URL: postgresql://${DB_USER}:${DB_PASSWORD}@db:5432/${DB_NAME}

  db:
    image: postgres:15
    environment:
      POSTGRES_USER: ${DB_USER}
      POSTGRES_PASSWORD: ${DB_PASSWORD}
      POSTGRES_DB: ${DB_NAME}
```

✓ Essa abordagem torna o mesmo `docker-compose.yml` reutilizável em dev, staging e produção, mudando apenas o arquivo `.env`.





Conectando com Plataformas de Deploy

Plataformas como **Railway** e **Render** simplificam o deploy de aplicações containerizadas, mas é importante entender o que acontece por baixo dos panos para depurar problemas e tomar boas decisões de arquitetura.



Railway — Como Funciona por Baixo

1

Push no Git

Você faz push para o repositório conectado ao Railway.

2

Build automático

Railway detecta o Dockerfile e executa o build. Sem docker-compose.yml em produção, cada serviço é deployado separadamente.

3

Serviços privados

O banco e outros serviços internos ficam em rede privada. Railway injeta a variável `DATABASE_URL` automaticamente.

4

DNS interno

Serviços se comunicam pelo hostname gerado pela plataforma, o mesmo conceito do DNS interno do Compose, mas gerenciado pelo Railway.

- ❑ O `docker-compose.yml` é a ferramenta de desenvolvimento. Em produção no Railway, você configura cada serviço individualmente pela dashboard ou pelo arquivo `railway.json`.



Render — O que Acontece por Baixo

Web Services

O Render detecta seu `Dockerfile` e faz o build. Você define as variáveis de ambiente pela dashboard, o equivalente ao seu `.env` local.

Banco de Dados

O Render oferece PostgreSQL gerenciado. Ele gera uma `DATABASE_URL` privada que você cola como variável de ambiente do seu backend, idêntico ao que você faz no Compose com o DNS interno.

O paralelo com o Compose

Compose: `depends_on`

Render: deploy em ordem manual ou via health checks da plataforma.

Compose: rede interna

Render: rede privada com hostname gerado automaticamente.

Compose: volumes nomeados

Render: discos persistentes configurados por serviço.



Checklist de Boas Práticas

1

Versione o docker-compose.yml

O arquivo deve estar no repositório. É a documentação executável do projeto.

2

Nunca commite o .env

Sempre adicione `.env` ao `.gitignore` e mantenha o `.env.example` atualizado.

3

Use healthchecks reais

Prefira `condition: service_healthy` no `depends_on` para garantir que o banco esteja pronto de fato.

4

Não exponha o banco para o host

Use `expose` em vez de `ports` para o PostgreSQL em ambientes que não sejam exclusivamente de desenvolvimento.



Recapitulando — O que Aprendemos

Compose resolve o caos

Um arquivo substitui dezenas de comandos manuais e garante reprodutibilidade total do ambiente.

DNS interno é poderoso

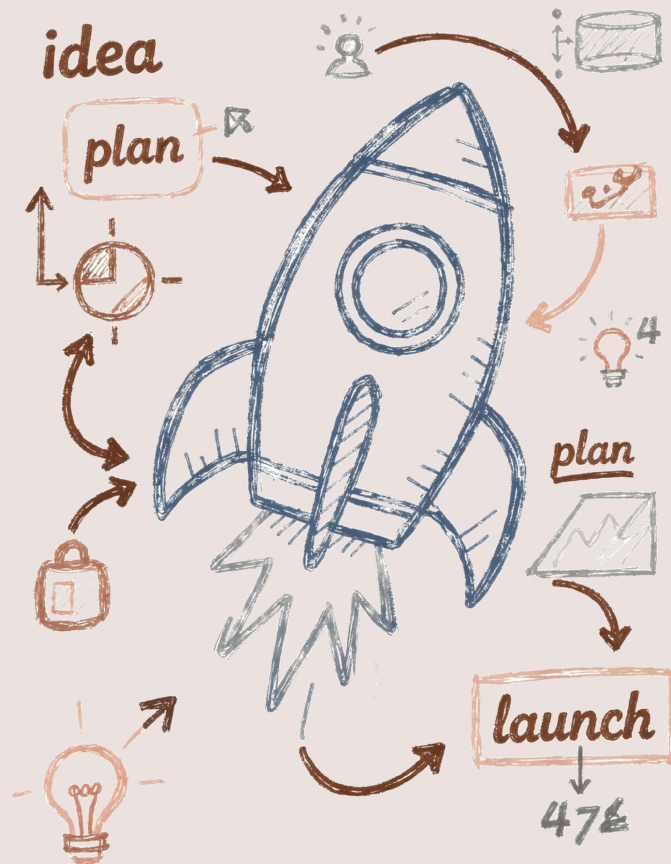
O nome do serviço é o hostname. Use-o nas connection strings em vez de IPs ou localhost.

Stop ≠ Down ≠ Down -v

Cada comando tem um impacto diferente nos dados. Conheça a diferença antes de usar em produção.

Plataformas replicam o conceito

Railway e Render implementam os mesmos princípios do Compose, redes privadas, variáveis de ambiente e ordem de inicialização.



Próximos Passos



Pratique agora

Clone o repositório do seu projeto, rode `docker compose up --build` e explore os logs de cada serviço.



Experimente o down -v

Suba o sistema, insira dados no banco, rode `docker compose down -v` e suba novamente. Observe o que acontece.



Faça o deploy

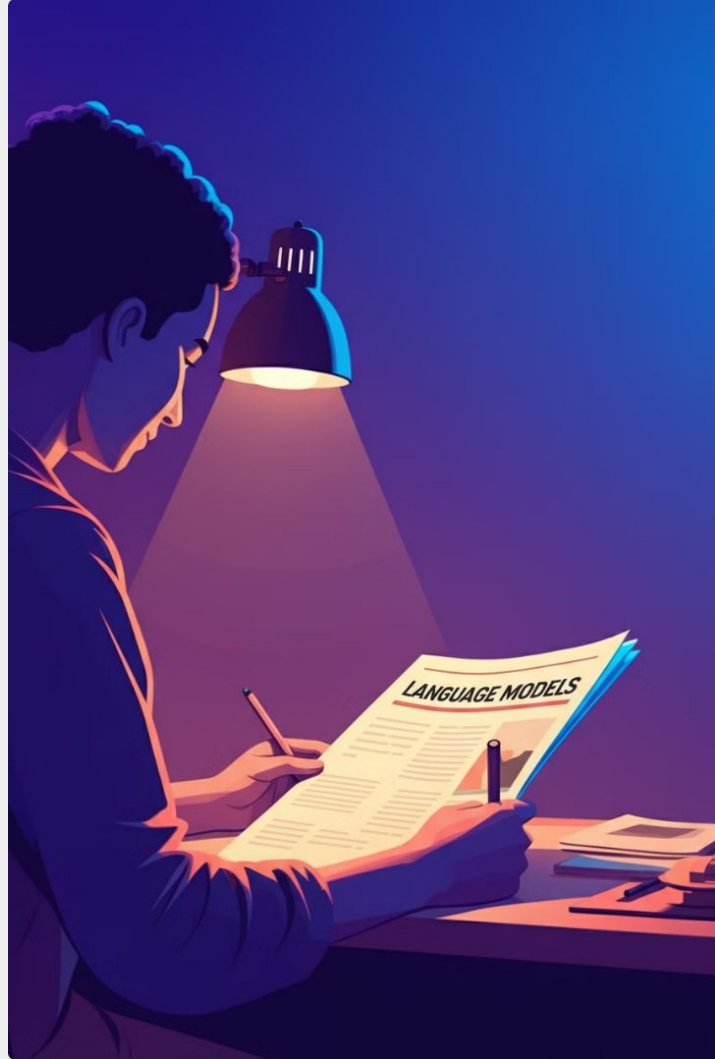
Conecte o repositório ao Railway ou Render, configure as variáveis de ambiente e veja seu sistema de três camadas rodando na nuvem.

- ✔ Você agora tem o conhecimento para orquestrar sistemas reais com Docker Compose. A fundação para entender Kubernetes e orquestradores avançados está construída. 🚀



Próximos passos

- Eng. Software em Dimensões
 - Cap. 2, seção 2.5.1 — Fator ônibus e socialização do conhecimento via Code Review.
 - Cap. 3 (seções 3.1–3.7): Versionamento, Git, PRs, Code Review, licenças
 - Cap. 14 (seções 14.1–14.8): Arquitetura, C4 Model, qualidade arquitetural
 - Cap. 19 (seções 19.1–19.4): Manutenção, refactoring, dívida técnica, documentação
- Eng. Software Moderna
 - Cap. 9 — Refactoring, Catálogo, code smells, prática
 - Cap. 10 — DevOps, Controle de versões, integração contínua



contacts

vcg@cin.ufpe.br
[@vinicius3w](https://twitter.com/vinicius3w)
viniciusgarcia.me

academy

go.uaite.xyz/vcg-es
go.uaite.xyz/vcg-microservices
go.uaite.xyz/vcg-devops
go.uaite.xyz/vcg-esaia
go.uaite.xyz/vcg-deso

Desenvolvimento de Software (CIN0136)

Orquestrando Serviços com
Docker Compose